

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: J Satya Srikar

Roll No.: A24126552259

1. TITLE

Working with Node.js Modules (CommonJS, In-built, Third-Party, and ES6)

2. AIM

To study and implement different Node.js module systems, including CommonJS module exports/imports, Node.js built-in modules (os, crypto), ES6 module syntax, and installation/usage of third-party npm modules (cowsay).

3. OBJECTIVE

The objectives of this experiment are:

- To study Node.js module loading mechanisms and module encapsulation formats.
 - To implement CommonJS module structures using module.exports and require() statements.
 - To utilize Node.js built-in core modules (os, crypto) for querying platform metadata and generating UUIDs.
 - To write modern ES6 syntax modules using standard import and export declarations.
 - To install, import, and configure third-party dependencies from npm registry (cowsay package).
-

4. THEORY

Node.js uses module structures to divide complex code into self-contained files. A module encapsulates its definitions, exposing selected components to other modules via explicit interface definitions. Node.js traditionally implements the CommonJS module system (using require() and module.exports) and natively supports the standardized ES6 Module system (using import and export statements). Additionally, Node.js includes core library modules out-of-the-box and provides npm to install third-party modular extensions.

Theory of Node.js Modules:

- **CommonJS Module System:** Traditional standard where each file is a module with local scope. Features are exported by assigning to the *module.exports* object, and loaded synchronously using the *require()* function.
 - **ES6 Module System (ESM):** The modern ECMAScript standard for JavaScript. Modules use the *export* keyword to expose components and the *import* keyword to load them. Files use *.mjs* or require "type": "module" configuration.
 - **Node.js In-built Modules:** Standard core libraries bundled inside Node.js. In this lab, we import *os* to inspect system properties (platform, free memory) and *crypto* to run cryptographic functions (e.g. *randomUUID()*).
 - **Third-Party Modules & NPM:** External packages created by developers. We use npm to initialize package scopes (*npm init*) and fetch modules (*npm install cowsay*), loading them via node resolution.
-

5. ALGORITHM / PROCEDURE

1. Establish project workspace folders: *Src/CommonJS*, *Src/InBuilt*, *Src/ThirdParty*, *Src/modernJS*, and *Docs*.
 2. In *Src/CommonJS*, write *`message.cjs`* to export custom functions, and *`mod.cjs`* to import and call them using *require*.
 3. In *Src/InBuilt*, create *`inbuilt.js`* importing Node.js core libraries (*os*, *crypto*) via ES6 import syntax.
 4. Implement system query statements (*os.platform()*, *os.freemem()*) and generate secure random hacker IDs (*crypto.randomUUID()*).
 5. Create duplicate ES6 import structures inside *`modernJS/inbuilt.js`* to demonstrate identical executions.
 6. In *Src/ThirdParty*, initialize npm project environment, install the *cowsay* package, and create *`others.js`* to invoke it.
 7. Run all modules inside terminal using Node.js, noting down system platform strings and cowsay ASCII layouts.
-

6. PROGRAM

Source Files: CommonJS Modules, Built-in Modules, and Third-Party Modules

1. CommonJS Module: message.cjs

```
const msg = () => {console.log("Hello from CommonJS module!");};
const add = (a,b) => {return a + b;};

module.exports = {msg, add};
```

2. CommonJS Consumer: mod.cjs

```
const mod1 = require('./message.cjs');
mod1.msg();
console.log(mod1.add(5, 10));
```

3. Node Built-in Modules: inbuilt.js

```
// hacker.mjs

// 1. Import the built-in modules
import os from 'os';
import crypto from 'crypto';

console.log("--- SYSTEM RECON ---");

// 2. Use 'os' to spy on your own computer
console.log("Operating System:", os.platform());
// Memory is in bytes, so we divide by 1024 twice to get Megabytes
console.log("Free RAM:", Math.round(os.freemem() / 1024 / 1024), "MB");

console.log("--- ENCRYPTION ---");

// 3. Use 'crypto' to generate a secure, random ID
const secretId = crypto.randomUUID();
console.log("Your Secret Hacker ID:", secretId);
```

4. Third-Party NPM Module: others.js

```
const cowsay = require('cowsay');
console.log(cowsay.say({
  text : "I am officially a Node.js developer now 🐮",
  e:"-O",
  T:"U "
}));
```

7. CODE EXPLANATION

Code Segment / Keyword	Explanation
<code>module.exports = {msg, add}</code>	Exports msg and add methods in a CommonJS module structure.
<code>const mod1 = require('./message.cjs')</code>	Loads local CommonJS message module using require keyword.
<code>import os from 'os'</code>	Loads Node.js built-in operating system library using ES6 import syntax.
<code>import crypto from 'crypto'</code>	Loads Node.js built-in cryptography library using ES6 import syntax.
<code>os.platform()</code>	Returns current operating system platform identifier (e.g. 'win32').
<code>os.freemem()</code>	Queries the operating system to retrieve current free system memory bytes.
<code>crypto.randomUUID()</code>	Generates and returns a secure, randomized version 4 UUID string.
<code>require('cowsay')</code>	Imports external cowsay dependency downloaded via npm package manager.
<code>cowsay.say({ text: ... })</code>	Formats text string inside ASCII cow character balloon graphics.

8. TEST CASES AND EXECUTION DATA

The following manual test cases verify correct execution of Node.js modules in a command prompt:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	CommonJS Imports	Run 'node Src/CommonJS/mod.cjs'	Prints greeting string and sum of values (15) correctly	Pass
TC-02	OS Platform Query	Run 'node Src/InBuilt/inbuilt.js'	Queries and prints hosting OS platform identifier	Pass
TC-03	Free RAM calculation	Run 'node Src/InBuilt/inbuilt.js'	Displays remaining physical memory formatted in MBs	Pass
TC-04	Crypto UUID Generation	Run 'node Src/InBuilt/inbuilt.js'	Generates and displays secure, random 36-char hacker ID	Pass
TC-05	Cowsay NPM Execution	Run 'node Src/ThirdParty/others.js'	Displays cow ASCII character saying the developer message	Pass

9. OUTPUT

Console Execution: CommonJS Modules (mod.cjs)

```
PS D:\FS-LAB\Record\Experiment5-week5> node Src\CommonJS\mod.cjs
Hello from CommonJS module!
15
```

Console Execution: InBuilt Core Modules (inbuilt.js)

```
PS D:\FS-LAB\Record\Experiment5-week5> node Src\InBuilt\inbuilt.js
--- SYSTEM RECON ---
Operating System: win32
Free RAM: 4648 MB
--- ENCRYPTION ---
Your Secret Hacker ID: b473d74d-ef2e-4dcb-bb22-677c9c0e3f3c
```

Console Execution: Third-Party NPM Module (others.js)

```
PS D:\FS-LAB\Record\Experiment5-week5> node Src\ThirdParty\others.js

_____
< I am officially a Node.js developer now ■ >
-----
      ^__^
      \  (-0)\_____
        (__)\\       )\\/\\
           U   ||----w |
               ||     ||
```

10. RESULT

Thus, different module loading systems in Node.js (CommonJS synchronous module files exports/imports, operating systems and cryptography core built-in libraries, ES6 module imports/exports, and third-party cowsay dependency modules) were successfully implemented, executed, and verified.

Submission Package Structure:

```
Experiment5-week5/  
  ■■■ Docs/  
  ■ ■■■ Experiment 5.pdf  
  ■■■ Src/  
  ■■■ CommonJS/  
  ■ ■■■ message.cjs  
  ■ ■■■ mod.cjs  
  ■ ■■■ package.json  
  ■■■ InBuilt/  
  ■ ■■■ inbuilt.js  
  ■■■ ThirdParty/  
  ■ ■■■ others.js  
  ■ ■■■ package-lock.json  
  ■ ■■■ package.json  
  ■■■ modernJS/  
  ■■■ inbuilt.js
```